

Remainder in Hashing Methods



are hash functions used for collision resolution?

Excellent question — and an important distinction in hashing!

♦ Short Answer:

No, hash functions are *not* used for collision resolution.

Hash functions **generate** the hash (index).

Collision resolution happens **after a collision occurs**.

♦ Clarifying the Roles

Concept	Purpose
Hash Function	Converts a key into a hash/index within the table (e.g., <code>key % table_size</code>)
Collision Resolution	Deals with situations where multiple keys hash to the same index

📌 What is a Collision?

A **collision** happens when:

```
cpp
```

```
hash("apple") == 5  
hash("banana") == 5
```

Now both want to go into index 5. That's a **collision** — and must be resolved.

🔧 Types of Collision Resolution

1. Open Addressing

- Try the next available slot (e.g., linear probing, quadratic probing)
- All values stay **within the hash table**

2. Chaining

- Each slot holds a **linked list** (or dynamic list) of items
- So, multiple items can live at the same index

So in summary:

Role	Is the hash function responsible?
Mapping keys to indexes	 Yes
Dealing with same-index conflicts	 No (collision resolution techniques handle this)

Analogy

Think of the **hash function** as a **ticketing machine** assigning seats.

If two people get the **same seat number**, it's a conflict.

Now someone (collision resolver) must decide:

- Should we seat one nearby?
- Should we add a foldable chair?
- Or ask one to share?

Hash function does the **assignment** — not the **conflict handling**.

Let me know if you'd like code examples of how both parts work!



I saw equations used for Linear probing, Quadratic probing and Double hashing. What are these equations called?

Great observation! The equations used for **Linear Probing**, **Quadratic Probing**, and **Double Hashing** are known as **probing functions** or **probing formulas**.

♦ What are they?

They are formulas used in **open addressing** to find the next available slot when a collision occurs.

Technique	Probing Formula	Name
Linear Probing	$h(k, i) = (h(k) + i) \% m$	Linear Probing Function
Quadratic Probing	$h(k, i) = (h(k) + c1 \cdot i + c2 \cdot i^2) \% m$	Quadratic Probing Function
Double Hashing	$h(k, i) = (h1(k) + i \cdot h2(k)) \% m$	Double Hashing Function

🧠 Explanation of Symbols:

- $h(k)$ or $h1(k)$: the **primary hash function**
- i : the **probe number** (0 for first attempt, 1 for next, etc.)
- m : size of the hash table
- $c1, c2$: constants (often $c1 = 1, c2 = 3$, etc.)
- $h2(k)$: a **secondary hash function** (for double hashing)

📌 Why Are They Called Probing Functions?

Because they define the "**sequence of positions**" you'll try **after a collision**. Each i gives a new position to probe.

For example:

- Linear probing just moves to the **next** slot
- Quadratic spreads out in **squares**
- Double hashing uses **another hash** to jump smartly

 Summary:

Type	Equation Name
Linear Probing	Linear Probing Function
Quadratic Probing	Quadratic Probing Function
Double Hashing	Double Hashing Function

Collectively, they are called **Probing Functions** in the context of **Open Addressing Hashing**.

Let me know if you want a visual or animation-style explanation of how they move in the table!